FACULTAD DE CIENCIAS
EXACTAS Y NATURALES
Y AGRIMENSURA

UNIVERSIDAD NACIONAL
DEL NORDESTE

Lic. en Sistemas de Información

"Arquitectura de software en entornos de computación en la nube"

Alumnos: Varela Franco Martín, Villalba Ruiz Diaz Renata.

Profesora: Gómez Solis, Maria de los Angeles Ferraro

Ciclo Lectivo: 2026

1. Introducción

La computación en la nube constituye un modelo de provisión de recursos informáticos orientado al acceso remoto, el aprovisionamiento bajo demanda, la elasticidad y la medición del uso. En las bibliografías utilizadas, este paradigma se presenta no sólo como una evolución tecnológica, sino también como una transformación arquitectónica y operativa que modifica la forma en que las organizaciones diseñan, despliegan, gestionan y escalan sus sistemas de software. Thomas Erl retoma la definición ampliamente aceptada de NIST, según la cual la nube habilita acceso ubicuo y conveniente a un conjunto compartido de recursos configurables que pueden aprovisionarse y liberarse rápidamente con mínimo esfuerzo de gestión; a su vez, resume el concepto como una forma especializada de computación distribuida orientada al aprovisionamiento remoto de recursos escalables y medidos.

El propósito del presente informe es analizar, desde una perspectiva académica y arquitectónica, los fundamentos de la arquitectura de software asociada a la computación en la nube, identificando sus conceptos estructurales, sus componentes tecnológicos, sus mecanismos operativos y las decisiones de diseño que la vuelven adecuada para responder a exigencias contemporáneas como escalabilidad, disponibilidad, seguridad, automatización y gobernanza. El alcance del análisis comprende tanto la arquitectura cloud en sentido general como los modelos de servicio y despliegue que la materializan, incluyendo IaaS, PaaS, SaaS, nube pública, privada, híbrida y multicloud. Asimismo, se incorpora una aproximación práctica apoyada en el documento de AWS para ejemplificar cómo estos principios se concretan en una plataforma real de servicios.

2. Desarrollo

2.1. Conceptos fundamentales

La computación en la nube no debe interpretarse únicamente como una infraestructura remota, sino como un ecosistema integral basado en servicios estandarizados y automatizados. Este modelo permite la provisión de recursos compartidos —como aplicaciones, capacidad de cómputo, almacenamiento, redes y plataformas de desarrollo— a través de mecanismos que priorizan la automatización y la eficiencia operativa. En este contexto, la automatización cumple un rol central, ya que posibilita el autoservicio por parte de los usuarios, la asignación y liberación dinámica de recursos dentro de un entorno compartido, y una gestión optimizada de la capacidad y de las cargas de trabajo.

En la misma línea, se identifican seis características principales presentes en la mayoría de los entornos cloud: uso bajo demanda, acceso ubicuo, multitenencia con pooling de recursos, elasticidad, uso medido y resiliencia. Estas características definen el valor arquitectónico del entorno, ya que permiten que los recursos se asignen y reasignen dinámicamente según demanda, que múltiples consumidores compartan infraestructura de manera aislada y que el consumo pueda ser monitoreado tanto para facturación como para control operativo.

Desde el punto de vista del negocio, AWS enfatiza ventajas complementarias: sustitución del gasto de capital por gasto variable, economías de escala, eliminación de la necesidad de prever capacidad con demasiada anticipación, mayor velocidad de aprovisionamiento, reducción de la carga operativa del centro de datos y capacidad de despliegue global en minutos. Estas ventajas no son ajenas a la arquitectura: derivan precisamente de una estructura técnica diseñada para abstraer infraestructura, automatizar provisión y distribuir recursos en distintas regiones y zonas de disponibilidad.

2.2. Modelos de servicio y despliegue

Dentro del paradigma de computación en la nube, los modelos de servicio se organizan en tres categorías principales: **Infraestructura como Servicio (IaaS)**, **Plataforma**

como Servicio (PaaS) y **Software como Servicio (SaaS).** Cada uno representa un distinto nivel de abstracción tecnológica y define qué componentes son gestionados por el proveedor y cuáles quedan bajo responsabilidad del usuario. En el modelo **IaaS**, se proveen recursos fundamentales como servidores virtuales, redes y almacenamiento, permitiendo al usuario configurar y administrar el sistema operativo y las aplicaciones según sus necesidades. En cambio, **PaaS** ofrece un entorno ya preparado que incluye el sistema operativo y herramientas de desarrollo, lo que simplifica el proceso de despliegue y reduce la carga operativa. Por último, **SaaS** entrega soluciones completas listas para su uso, donde el proveedor se encarga de toda la gestión técnica y el usuario solo interactúa con la aplicación final.

Desde una mirada arquitectónica, estos modelos pueden entenderse como una evolución progresiva en la externalización de responsabilidades. A medida que se avanza de IaaS a **SaaS**, disminuye el nivel de control directo del usuario sobre la infraestructura, pero aumenta la simplicidad en la gestión y el enfoque en la funcionalidad. Este enfoque permite a las organizaciones elegir el modelo más adecuado según sus capacidades técnicas, necesidades operativas y objetivos estratégicos.

En cuanto a las estrategias de despliegue, existen diferentes formas de implementar soluciones en la nube. La nube pública se caracteriza por el uso de infraestructura compartida provista por terceros, mientras que la nube privada se destina exclusivamente a una organización, ofreciendo mayor control y aislamiento. La nube híbrida combina ambos entornos, integrando recursos locales con servicios en la nube para lograr mayor flexibilidad y adaptabilidad. Finalmente, el enfoque multicloud implica el uso coordinado de múltiples proveedores de nube, lo que permite diversificar recursos, evitar dependencia de un único proveedor y optimizar el rendimiento y los costos, aunque introduce mayores desafíos en términos de integración y gestión.

2.3. Arquitectura de software cloud: componentes y mecanismos

La arquitectura cloud puede entenderse a partir de un conjunto de mecanismos tecnológicos que actúan como piezas fundamentales para construir soluciones más amplias. Estos mecanismos incluyen elementos de infraestructura como el perímetro lógico de red, el servidor virtual, los dispositivos de almacenamiento en la nube, los monitores de uso, la replicación de recursos y los entornos preconfigurados. A ellos se suman mecanismos más especializados, entre ellos el escuchador de escalado automático, los balanceadores de carga, los monitores de niveles de servicio, los monitores de pago por uso y auditoría, los

sistemas de conmutación por error, los hipervisores, los clústeres de recursos y las bases de datos para la gestión del estado.

Este enfoque concibe la nube como un sistema modular compuesto por componentes que cumplen funciones específicas, en lugar de una estructura rígida y única. Cada mecanismo aporta una capacidad concreta: aislamiento, provisión de recursos, supervisión, escalado, distribución de carga, recuperación ante fallos y protección del servicio. El perímetro lógico de red, por ejemplo, establece una frontera virtual que separa recursos distribuidos; el escuchador de escalado automático ajusta la capacidad según la demanda; los balanceadores reparten el trabajo entre distintas instancias; y los mecanismos de monitoreo y auditoría garantizan métricas confiables, cumplimiento y control de calidad.

Esta visión modular aparece en otro lenguaje, pero con la misma lógica: la nube se organiza sobre *resource pools* que alimentan servicios de infraestructura, plataforma y aplicación, mientras en la capa superior se ubican servicios de gestión, seguridad, gobernanza, monitoreo, integración y automatización. Se remarca que, para que el *resource pooling* funcione, cada elemento debe construirse como servicio independiente, con interfaces bien definidas, y que la arquitectura multitenant permite compartir recursos sin perder aislamiento de datos y configuración.

2.4. Patrones arquitectónicos predominantes

Aunque no se referencie la palabra “patrón”, sí describen patrones arquitectónicos reconocibles, entre ellas: **workload distribution**, **resource pooling**, **dynamic scalability**, **elastic resource capacity**, **service load balancing**, **cloud bursting**, **redundant storage**, **zero downtime** y **dynamic failure detection and recovery**. Estas arquitecturas revelan una orientación clara hacia la distribución, la redundancia, la elasticidad y la continuidad operativa.

Además, se incorpora el enfoque **cloud native**, basado en microservicios, contenedores, orquestación y APIs. Allí se sostiene que las aplicaciones cloud native se descomponen en servicios modulares e independientes, interactúan a través de contratos bien definidos y pueden descubrirse, desplegarse y escalarse de manera consistente. Esta aproximación constituye un patrón arquitectónico contemporáneo que refuerza la mantenibilidad, la resiliencia y la portabilidad.

A nivel práctico, AWS ejemplifica varios de estos patrones mediante servicios concretos: EC2 y Auto Scaling materializan elasticidad y escalado automático; Elastic Load Balancing implementa distribución de carga; Lambda expresa el modelo event-driven sin

administración de servidores; S3 representa almacenamiento de objetos altamente durable e integrado; VPC y grupos de seguridad materializan aislamiento lógico y control de acceso.

Estas arquitecturas y patrones se centran en cómo diseñar sistemas que sean resistentes, capaces de crecer según la demanda y que funcionen de manera eficiente en la nube. Se pueden agrupar en tres grandes pilares:

1. Disponibilidad y Continuidad Operativa

Estos patrones buscan que el sistema nunca deje de funcionar, incluso si ocurren fallos.

- **Redundancia y Almacenamiento Redundante:** Consiste en duplicar componentes o datos. Si una unidad de almacenamiento falla, hay otra copia exacta lista para entrar en funcionamiento sin pérdida de información (como sucede en **Amazon S3**).
- **Zero Downtime (Tiempo de inactividad cero):** Estrategias para actualizar o mantener el sistema sin que el usuario note una interrupción.
- **Dynamic Failure Detection and Recovery:** El sistema monitoriza su propio estado. Si detecta que un componente ha fallado, lo reinicia o lo reemplaza automáticamente.
- **Cloud Bursting:** Permite que una aplicación que corre en una infraestructura privada "se desborde" hacia una nube pública cuando la demanda supera la capacidad local, evitando que el servicio caiga por saturación.

2. Elasticidad y Gestión de Recursos

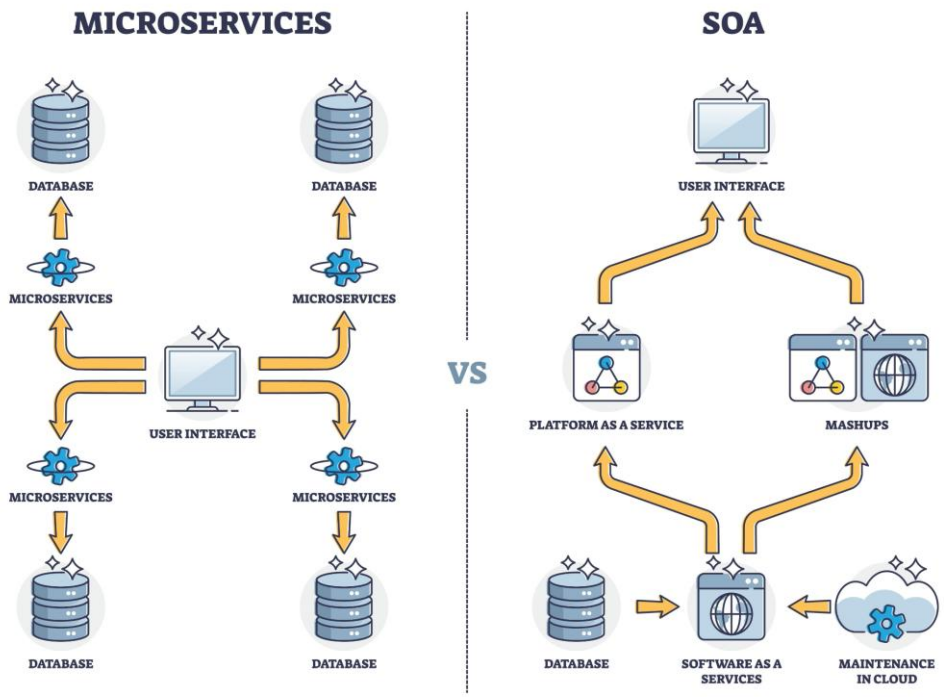
Aquí el foco es el uso eficiente del dinero y la computación: usar solo lo que se necesita.

- **Dynamic Scalability & Elastic Resource Capacity:** Es la capacidad de sumar o restar potencia (CPU, RAM) automáticamente. En AWS, esto se ve con **Auto Scaling**: si hay muchos usuarios, se crean más servidores; si baja el tráfico, se apagan para ahorrar costos.
- **Workload Distribution & Service Load Balancing:** Se utiliza un "distribuidor" (**Load Balancer**) que recibe todas las peticiones y las reparte equitativamente entre varios servidores para que ninguno se colapse.
- **Resource Pooling:** Agrupar recursos físicos para que se asignen dinámicamente a diferentes tareas según sea necesario, optimizando el hardware.

3. Arquitecturas Modernas (Cloud Native y Serverless)

Es el enfoque contemporáneo para desarrollar software que sea fácil de mantener y desplegar.

- **Microservicios:** En lugar de una aplicación gigante (monolito), el software se divide en piezas pequeñas e independientes (ej. un servicio para pagos, otro para el catálogo). Esto mejora la **mantenibilidad**, ya que puedes actualizar una parte sin romper el resto.
- **Contenedores y Orquestación:** Los microservicios se empaquetan en contenedores (como Docker) para que funcionen igual en cualquier computadora, y se gestionan con orquestadores para controlar su despliegue.
- **Arquitectura Event-Driven (Basada en eventos):** El sistema reacciona a acciones específicas. Por ejemplo, cuando un usuario sube una foto, se dispara una función (**AWS Lambda**) que la procesa. No hay un servidor encendido todo el tiempo esperando; solo se ejecuta cuando ocurre el "evento".
- **Aislamiento Lógico (VPC):** Aunque estés en una nube pública, estos patrones crean una "burbuja" privada para tu red, controlando quién entra y sale mediante reglas de seguridad.



2.5. Flujo de datos y de procesamiento

El flujo de datos en una infraestructura de nube moderna se estructura a través de **servicios desacoplados**, los cuales garantizan la escalabilidad y el monitoreo constante del sistema. En una implementación típica sobre **PaaS (Platform as a Service)**, la lógica se fragmenta en componentes de *front-end* y *back-end*; mientras el primero gestiona la interacción directa con el usuario, el segundo procesa las tareas de larga duración o de cómputo intensivo. Ambos niveles convergen en una capa de **almacenamiento persistente compartido**, consolidando una arquitectura por capas con una clara separación de responsabilidades.

Bajo el paradigma **Cloud Native**, este procesamiento se apoya en el uso de **microservicios, contenedores y APIs**. La comunicación entre módulos se realiza mediante interfaces estandarizadas, lo que permite un despliegue independiente, una mayor reutilización de código y una integración fluida entre componentes. En entornos **híbridos o multicloud**, la arquitectura debe ser lo suficientemente flexible para administrar ciclos de vida dinámicos, permitiendo que las cargas de trabajo y los datos se desplacen entre diferentes regiones o proveedores según variables críticas como el costo, la latencia, la confiabilidad y las normativas de negocio.

Finalmente, el ecosistema se potencia mediante un **flujo distribuido y orientado a eventos**. En este esquema, los dispositivos periféricos pueden recolectar información que es procesada de forma inmediata mediante computación *serverless*, almacenada en bases de datos NoSQL de alto rendimiento y, posteriormente, explotada por motores analíticos. Un componente esencial en este flujo es el **Edge Computing**, que permite filtrar y procesar datos en el borde de la red; esto optimiza el ancho de banda al transmitir hacia la nube central únicamente la información relevante, estableciendo así un sistema de procesamiento jerárquico y altamente eficiente.

2.6. Seguridad, gobierno y continuidad operativa

La arquitectura cloud incorpora seguridad como dimensión transversal. Se destacan amenazas como autorización insuficiente, conflictos entre límites de confianza, ataques de virtualización y disparidad de políticas de seguridad entre proveedor y consumidor; de allí la necesidad de mecanismos de cifrado, hashing, firmas digitales, IAM, SSO, grupos de seguridad e imágenes endurecidas. También enfatiza que la evaluación de riesgos debe integrarse al proceso de adopción y selección del proveedor.

AWS formula esta cuestión mediante el principio de **responsabilidad compartida**: el proveedor protege la seguridad de la nube y el cliente protege la seguridad en la nube. Esta distinción es arquitectónicamente relevante porque delimita responsabilidades sobre infraestructura, plataformas, configuraciones, identidades, cifrado y datos. Asimismo, AWS remarca que su infraestructura global basada en regiones y zonas de disponibilidad aisladas refuerza tolerancia a fallos, disponibilidad y escalabilidad.

Finalmente, se insiste en que la seguridad y la gobernanza son indispensables especialmente en arquitecturas híbridas y multicloud, donde la integración entre entornos introduce complejidad adicional. También resalta la relevancia de estándares para evitar dependencia del proveedor, facilitar interoperabilidad y hacer más simple la integración entre nubes públicas, privadas y centros de datos locales.

3. Ejemplo de aplicación práctica

3.1. Planteamiento de la consigna

Se propone como caso hipotético una **plataforma de telemedicina y monitoreo remoto de pacientes crónicos**. La aplicación permite que médicos y pacientes interactúen mediante videollamadas, consultas asíncronas, historial clínico digital y recepción de datos provenientes de dispositivos IoT domiciliarios, como tensiómetros, oxímetros y glucómetros conectados.

Requisitos funcionales

1. El sistema debe permitir autenticación diferenciada para médicos, pacientes y administradores.
2. El sistema debe recibir, almacenar y visualizar datos biométricos enviados por dispositivos conectados.
3. El sistema debe generar alertas automáticas cuando los valores reportados superen umbrales clínicos definidos.
4. El sistema debe ofrecer acceso web y móvil al historial de consultas, estudios y medicación.
5. El sistema debe soportar videoconferencias seguras entre paciente y profesional.

Requisitos no funcionales

1. **Escalabilidad:** la plataforma debe soportar crecimiento de usuarios, sesiones simultáneas y volumen variable de eventos IoT.
2. **Seguridad y conformidad:** debe proteger datos sensibles, aplicar controles de acceso estrictos, cifrado y trazabilidad.
3. **Mantenibilidad:** debe permitir evolución modular del sistema sin afectar el servicio completo.
4. **Alta disponibilidad:** el sistema debe minimizar interrupciones y tolerar fallos de componentes.

3.2. Justificación arquitectónica

La arquitectura cloud analizada en los documentos responde de forma especialmente adecuada a este caso de uso. En primer lugar, una **arquitectura híbrida o multicloud** es razonable porque algunos datos clínicos sensibles o integraciones con sistemas hospitalarios heredados pueden permanecer en un entorno privado o controlado, mientras que los servicios de interacción, analítica o escalado de picos pueden desplegarse en nube pública. Esta distribución coincide con la idea de que ciertos workloads requieren proximidad con sistemas legados o mayor control, mientras otros pueden beneficiarse de recursos públicos más flexibles.

En segundo lugar, se determina que un **enfoque modular basado en servicios** resulta la solución óptima para los objetivos planteados. Bajo este esquema, la plataforma se descompone en **microservicios independientes** encargados de tareas críticas como la autenticación, la gestión de pacientes, la ejecución de videollamadas, la ingesta de datos provenientes de dispositivos IoT, el motor de alertas y la analítica avanzada.

Esta arquitectura se alinea con los principios fundamentales de las **aplicaciones nativas de la nube (Cloud Native)**, donde los servicios operan de forma autónoma y se comunican a través de contratos de **APIs bien definidos**. La implementación de esta estructura no solo mejora la **mantenibilidad** del sistema a largo plazo, sino que también facilita ciclos de **integración y despliegue continuo (CI/CD)**, permitiendo escalar componentes de manera individual y minimizando el riesgo de regresiones o impactos colaterales ante modificaciones en módulos específicos

En tercer lugar, los **mecanismos cloud** definidos por Erl permiten justificar la robustez de la solución. El *logical network perimeter* y los grupos de seguridad aíslan los componentes críticos; los *load balancers* distribuyen solicitudes de consultas y accesos concurrentes; el *automated scaling listener* y las arquitecturas de escalabilidad dinámica amplían capacidad

frente a picos de uso; la replicación de recursos y los esquemas de *failover* fortalecen la continuidad del servicio; y los monitores de SLA y uso medido permiten verificar rendimiento, disponibilidad y consumo.

Desde la perspectiva de seguridad, la arquitectura también es consistente. El principio de responsabilidad compartida señalado por AWS se adapta bien a una solución sanitaria: el proveedor puede garantizar la seguridad física y de base de la plataforma, mientras la organización sanitaria conserva el control sobre identidades, permisos, cifrado de datos y cumplimiento normativo. Además, servicios de IAM, cifrado, administración de claves y auditoría respaldan la protección del dato clínico.

Si se incorpora monitoreo remoto con dispositivos, los servicios orientados a eventos y borde también resultan pertinentes. El documento de AWS muestra que una combinación de IoT, Lambda y servicios de almacenamiento/analítica permite recibir datos desde dispositivos, procesarlos automáticamente y generar acciones sin administrar infraestructura compleja. Para un sistema de salud, esto permite transformar señales biométricas en alertas o recomendaciones clínicas con baja latencia y escalado flexible.

Por todo ello, la arquitectura cloud detallada en las fuentes no solo satisface los requisitos planteados, sino que lo hace con una estructura técnicamente superior a una solución monolítica on-premise tradicional. La nube aporta elasticidad, distribución geográfica, automatización, medición, seguridad por capas y modularidad, cualidades especialmente valiosas en aplicaciones con alta variabilidad de demanda y datos sensibles.

---

4. Conclusión

El análisis de los documentos permite concluir que la arquitectura de software en la computación en la nube se fundamenta en una combinación de principios de distribución, automatización, abstracción de recursos, medición del consumo, seguridad transversal y capacidad de adaptación dinámica. La nube no constituye solamente un entorno de hosting, sino una arquitectura integral basada en características como uso bajo demanda, multitenencia, elasticidad y resiliencia, implementadas mediante mecanismos específicos y patrones de composición como escalado dinámico, balanceo de carga, redundancia y distribución de cargas.

Asimismo, los modelos IaaS, PaaS y SaaS evidencian distintos niveles de abstracción arquitectónica, mientras que los despliegues híbridos y multicloud demuestran que la arquitectura cloud contemporánea ya no se limita a una única plataforma, sino que integra múltiples entornos según criterios de negocio, cumplimiento, rendimiento y portabilidad. En ese sentido, los estándares, la gestión de workloads y la modularidad basada en servicios y APIs se vuelven esenciales para evitar dependencia excesiva del proveedor y sostener la evolución del sistema en el tiempo.

Como reflexión final, puede afirmarse que la arquitectura estudiada presenta una **alta viabilidad y robustez frente a las demandas tecnológicas actuales**. Su fortaleza radica en su capacidad para responder de manera simultánea a exigencias de escalabilidad, disponibilidad, integración, gobernanza y seguridad. No obstante, los documentos también advierten que su éxito depende de decisiones arquitectónicas informadas, evaluación de riesgos, selección adecuada del modelo de despliegue y comprensión clara de las responsabilidades entre consumidor y proveedor. En consecuencia, la nube no debe asumirse como una solución automática, sino como un marco arquitectónico potente cuya efectividad depende de un diseño riguroso y de una adopción estratégicamente planificada.

5. Bibliografía

- Erl, T., Puttini, R., & Mahmood, Z. (2013). Cloud computing: Concepts, technology & architecture. Prentice Hall.
- Daniel Kirsch, J. H. (2020). Cloud Computing for dummies. John Wiley & Sons.